

A APPENDIX

A.1 Omitted Algorithms

Algorithm 9: BS-BiBFS

```

Input   :  $G = (V, E)$ : a DSG,  $Q(s, t, k, \sigma)$ : a query
Output  : true / false
1  $Q_1 \leftarrow \{(\{s\}, +)\}$ ,  $Q_2 \leftarrow \{(\{t\}, +)\}$ ,  $d_1 \leftarrow 0$ ,  $d_2 \leftarrow 0$ ;
2 Initialize  $M_1, M_2$  as empty HashMaps;
3  $M_1[s] \leftarrow \{(\{s\}, +, 0)\}$ ,  $M_2[t] \leftarrow \{(\{t\}, +, 0)\}$ ;
4 while  $(Q_1 \cup Q_2) \neq \emptyset \wedge (d_1 + d_2) < k$  do
5    $Next\_Q_1 \leftarrow \emptyset$ ;
6   for each  $(p_1, \sigma_{p_1}) \in Q_1$  do
7      $u \leftarrow$  the last vertex in  $p_1$ ;
8     for each  $(v, \sigma_{uv}) \in N_{out}(u)$  do
9       if  $v \notin p_1$  then
10        if  $v = t \wedge \sigma_{p_1} \cdot \sigma_{uv} = \sigma \wedge d_1 + 1 \leq k$  then
11          return true;
12        if  $v \in M_2.keys$  then
13          for each  $(p_2, \sigma_{p_2}, len_2) \in M_2[v]$  do
14            if  $(d_1 + 1 + len_2 \leq k) \wedge (\sigma_{p_1} \cdot \sigma_{uv} \cdot \sigma_{p_2} =$ 
15               $\sigma) \wedge (p_1 \cap p_2 = \emptyset)$  then
16              return true;
16          Store  $(p_1 \cup \{v\}, \sigma_{p_1} \cdot \sigma_{uv})$  into  $Next\_Q_1$ ;
17          Store  $(p_1 \cup \{v\}, \sigma_{p_1} \cdot \sigma_{uv}, d_1 + 1)$  into  $M_1[v]$ ;
18    $Q_1 \leftarrow Next\_Q_1$ ,  $d_1 \leftarrow d_1 + 1$ ;
19    $Next\_Q_2 \leftarrow \emptyset$ ;
20   for each  $(p_2, \sigma_{p_2}) \in Q_2$  do
21      $w \leftarrow$  the last vertex in  $p_2$ ;
22     for each  $(v, \sigma_{vw}) \in N_{in}(w)$  do
23       if  $v \notin p_2$  then
24        if  $v = s \wedge \sigma_{vw} \cdot \sigma_{p_2} = \sigma \wedge d_2 + 1 \leq k$  then
25          return true;
26        if  $v \in M_1.keys$  then
27          for each  $(p_1, \sigma_{p_1}, len_1) \in M_1[v]$  do
28            if  $(len_1 + d_2 + 1 \leq k) \wedge (\sigma_{p_1} \cdot \sigma_{vw} \cdot \sigma_{p_2} =$ 
29               $\sigma) \wedge (p_1 \cap p_2 = \emptyset)$  then
30              return true;
30          Store  $(p_2 \cup \{v\}, \sigma_{vw} \cdot \sigma_{p_2})$  into  $Next\_Q_2$ ;
31          Store  $(p_2 \cup \{v\}, \sigma_{vw} \cdot \sigma_{p_2}, d_2 + 1)$  into  $M_2[v]$ ;
32    $Q_2 \leftarrow Next\_Q_2$ ,  $d_2 \leftarrow d_2 + 1$ ;
33 return false;

```

BS-BiBFS. In the literature, online methods for reachability queries typically extend the breadth-first search (BFS) paradigm (e.g., [14, 43, 60]), which explores the graph layer by layer, maintaining a global visited vertex set (or two in bidirectional BFS). To answer the HSR problem, the traditional BFS framework must be extended to incorporate both sign and simple path constraints. A key difference lies in the traversal semantics: unlike traditional BFS, where each vertex is visited at most once, our problem setting allows revisiting the same vertex through multiple distinct paths. Consequently, we need to explicitly maintain all partial paths under exploration, including path sign and full vertex sequences within it. At each expansion step, we need to verify whether the newly visited vertex has already appeared in the current path, and compute the sign of path when necessary. To mitigate the search depth, this method can be extended to a bidirectional BFS-based strategy. This variant performs simultaneous forward and backward searches, determining reachability by identifying intersecting frontiers that satisfy both sign and simple path constraints. We refer to this method as

a baseline named BS-BiBFS, whose pseudocode is illustrated in Algorithm 9. Here a path p is a vertex sequence; we write $p \cup \{v\}$ for the path obtained by appending v to p , and use p for its vertex set in set operations such as $v \notin p$ and $p_1 \cap p_2$. BS-BiBFS maintains two queues, Q_1 and Q_2 , to manage the candidate paths for the forward and backward expansions, respectively. Additionally, it employs two hash maps, M_1 and M_2 , to index the visited paths by their endpoints (lines 1-3). In each iteration, BS-BiBFS executes the following steps. The first part is forward expansion (Lines 5-18), which extracts a path p_1 from Q_1 and extends it to an outgoing neighbor v . It enforces the simple path constraint ($v \notin p_1$) and checks for direct reachability to t . It performs a meeting check in Lines 12-15: if v has been visited by the backward search (i.e., $v \in M_2.keys$), the algorithm retrieves the backward paths p_2 from $M_2[v]$. It then verifies whether the concatenated path satisfies the length constraint, the sign constraint, and the disjoint constraint (Lines 13-15). If valid, it returns true; otherwise, the extended path is stored in the next frontier and recorded in $M_1[v]$. The other part is the backward expansion (Lines 19-32), which is similar to the forward expansion and we omit its description here. BS-BiBFS terminates when a valid path is found during either expansion phase, or returns false after exhausting the search space. The time complexity of BS-BiBFS is bounded by $O(k \cdot |V|^k)$. While this method guarantees correctness, maintaining all explored paths imposes a significant memory burden and efficiency bottleneck with both the hop constraint and the graph size.

BS-A*. By extending the A* search algorithm, we propose a baseline named BS-A*, details are described in Algorithm 10. Generally, BS-A* operates in two phases: a heuristic generation phase and a guided search phase. Since computing the exact shortest signed simple path is NP-hard, in the first phase, BS-A* adopts a relaxed strategy to guide the search efficiently. It first executes a reverse BFS-based search from the target vertex t to compute the shortest signed distances (ignoring the simple path constraint) in Lines 24-36. Generally, for each vertex $v \in V$, $D_{pos}[v]$ (resp. $D_{neg}[v]$) denotes the shortest positive (resp. negative) distance from v to t by ignoring the simple path constraint. This pre-computation serves two critical purposes: it provides an admissible heuristic for the subsequent forward search, and it enables early termination, i.e., if the source vertex s cannot reach t even via a path within the hop constraint k , the algorithm immediately returns false. The second phase employs a forward search from s guided by the heuristic values computed in the first phase, iteratively exploring paths prioritized by their estimated total cost $f(\cdot)$. At each step, it extracts the most promising state from the priority queue PQ and extends the path while strictly enforcing the simple path constraint by maintaining a visited vertex set P within each state tuple. The search terminates when a valid s - t simple path matching the sign constraint is found (Lines 10-11), or when the priority queue is exhausted without success. The time complexity of BS-A* is bounded by $O(k \cdot |V|^k)$. In a nutshell, this method actively utilizes the pre-computed distance and sign data to prune the search space; search branches that are theoretically unreachable or exceed the hop constraint k are discarded early, thereby avoiding unnecessary computation in unpromising subspaces. However, due to its failure to account for the simple path constraint, its priority mechanism may provide misleading

Algorithm 10: BS-A*

```

Input :  $G = (V, E)$ : a DSG,  $Q(s, t, k, \sigma)$ : a query
Output : true / false
1  $(D_{pos}, D_{neg}) \leftarrow \text{Rev-Signed-BFS}(G, t, k)$ ;
2 if  $\sigma = +$  then  $h \leftarrow D_{pos}[s]$ ;
3 if  $\sigma = -$  then  $h \leftarrow D_{neg}[s]$ ;
4 if  $h > k$  then return false;
5  $PQ \leftarrow \emptyset$ ,  $PQ.\text{push}((h, 0, s, +, \{s\}))$ ;
6 while  $PQ \neq \emptyset$  do
7    $(f, g, u, \sigma_1, P) \leftarrow PQ.\text{pop}()$ ; /* Pop state with min  $f$ -score */
8   for each  $(v, \sigma_{uv}) \in N_{out}(u)$  do
9     if  $v \notin P$  then
10      if  $v = t \wedge \sigma_1 \cdot \sigma_{uv} = \sigma \wedge g + 1 \leq k$  then
11        return true;
12      if  $v = t$  then
13        continue;
14       $g_{new} \leftarrow g + 1$ ;
15      if  $g_{new} \leq k$  then
16         $\sigma_{new} \leftarrow \sigma_1 \cdot \sigma_{uv}$ ;
17         $\sigma_{rem}$  is computed to satisfy  $\sigma_{new} \cdot \sigma_{rem} = \sigma$ ;
18        if  $\sigma_{rem} = +$  then  $h_{val} \leftarrow D_{pos}[v]$ ;
19        if  $\sigma_{rem} = -$  then  $h_{val} \leftarrow D_{neg}[v]$ ;
20        if  $h_{val} \neq \infty \wedge (g_{new} + h_{val} \leq k)$  then
21           $f_{new} \leftarrow g_{new} + h_{val}$ ;
22           $PQ.\text{push}((f_{new}, g_{new}, v, \sigma_{new}, P \cup \{v\}))$ ;
23 return false;

24 Procedure Rev-Signed-BFS( $G, t, k$ )
25 Initialize  $D_{pos}[\cdot] \leftarrow \infty, D_{neg}[\cdot] \leftarrow \infty$ ;
26  $D_{pos}[t] \leftarrow 0, Q \leftarrow \{(t, 0, +)\}$ ;
27 while  $Q \neq \emptyset$  do
28    $(u, dist, \sigma_{end}) \leftarrow Q.\text{dequeue}()$ ;
29   if  $dist \geq k$  then continue;
30   for each  $(v, \sigma_{vu}) \in N_{in}(u)$  do
31      $\sigma_{req}$  is computed to satisfy  $\sigma_{req} \cdot \sigma_{vu} = \sigma_{end}$ ;
32     if  $\sigma_{req} = + \wedge D_{pos}[v] = \infty$  then
33        $D_{pos}[v] \leftarrow dist + 1, Q.\text{enqueue}((v, dist + 1, +))$ ;
34     else if  $\sigma_{req} = - \wedge D_{neg}[v] = \infty$  then
35        $D_{neg}[v] \leftarrow dist + 1, Q.\text{enqueue}((v, dist + 1, -))$ ;
36 return  $(D_{pos}, D_{neg})$ ;

```

guidance by suggesting search directions that are actually invalid (forming cycles).

ExPathD-I in LV0-I. The pseudocode of ExPathD-I (in LV0-I) is presented in Algorithm 11. In general, it first computes and stores the candidate incoming neighbors for each vertex (Lines 1-10), and then invokes the recursive procedure Sub-DFS to obtain the result (Lines 11-15). In Sub-DFS, the variable *Flag* indicates whether a valid path has been found and is initialized to -1 . During reverse traversal, several pruning rules (Lines 20-22) are applied to eliminate invalid or redundant explorations.

ExPathD-II in LV0-II. The pseudocode of ExPathD-II (in LV0-II) is presented in Algorithm 12. It follows the same framework as Algorithm 11 but with one more pruning rule (Lines 6-9). The correctness of PR1-PR4 is formally established in Appendix A.2 (Lemma A.6). The examples of PR1-PR4 are illustrated in Appendix A.3 (Example A.9-A.10).

A.2 Proofs and omitted Theorems

PROOF OF LEMMA 4.5. Suppose FaPathD detect the path $\langle (s = v_0, v_1, \sigma_1), (v_1, v_2, \sigma_2), \dots, (v_{i-1}, v_i = t, \sigma_i) \rangle$. First, it can be verified

Algorithm 11: ExPathD-I ($s, t, k, \sigma, \text{reach}$)

```

Output : true / false
/* Stage I:  $CN_{in}^{+/-}(\cdot)$  sets computation */
1 for each  $\sigma_v \in \{+, -\}$  do
2   for  $v \in V$  do
3      $CN_{in}^{\sigma_v}(v) \leftarrow \emptyset$ ;
4     for each  $(u, \sigma_{uv}) \in N_{in}(v)$  do
5        $\sigma_u$  is computed to satisfy  $\sigma_u \cdot \sigma_{uv} = \sigma_v$ ;
6       if  $\text{reach}_u^{\sigma_u} = -1$  then
7         continue;
8       else
9          $CN_{in}^{\sigma_v}(v).\text{push}(\text{reach}_u^{\sigma_u}, u, \sigma_{uv})$ ;
10    sort  $CN_{in}^{\sigma_v}(v)$  in ascending order of reach values;

/* Stage II: reverse traversal with pruning rules */
11 for each  $z \in V$  do
12    $\text{visit}[z] \leftarrow \text{false}$ ;
13 if Sub-DFS( $s, t, \sigma, 0, k$ ) = 1 then
14   return true;
15 else return false;

16 Procedure Sub-DFS( $s, w, \sigma_w, h, k$ )
17 if  $s = w$  then return 1;
18  $\text{Flag} \leftarrow -1, \text{visit}[w] \leftarrow \text{true}$ ;
19 for each  $(d_v, v, \sigma_{vw}) \in CN_{in}^{\sigma_w}(w)$  do
20   if  $h = k - 1 \wedge v \neq s$  then continue; /* PR1 */
21   if  $d_v + h \geq k$  then continue; /* PR2 */
22   if  $\text{visit}[v]$  then continue; /* PR3 */
23    $\sigma_v$  is computed to satisfy  $\sigma_v \cdot \sigma_{vw} = \sigma_w$ ;
24   if Sub-DFS( $s, v, \sigma_v, h + 1, k$ ) = 1 then
25      $\text{Flag} \leftarrow 1, \text{break}$ ;
26  $\text{visit}[w] \leftarrow \text{false}$ ;
27 return  $\text{Flag}$ ;

```

Algorithm 12: ExPathD-II ($s, t, k, \sigma, \text{reach}, SV$)

```

Output : true / false
1 Same as Lines 1-15 of Algorithm 11;

2 Procedure Sub-DFS( $s, w, \sigma_w, h, k$ )
3 Same as Lines 17-18 of Algorithm 11;
4 for each  $(d_v, v, \sigma_{vw}) \in CN_{in}^{\sigma_w}(w)$  do
5   Same as Lines 20-23 of Algorithm 11;
6    $F_{dup} \leftarrow \text{false}, d' \leftarrow k - h - 1$ ;
7   for each  $x \in SV_{d'}^{\sigma_v}(v)$  do
8     if  $\text{visit}[x]$  then  $F_{dup} \leftarrow \text{true}, \text{break}$ ;
9   if  $F_{dup}$  = true then continue; /* PR4 */
10  Same as Lines 24-25 of Algorithm 11;
11 Same as Lines 26-27 of Algorithm 11;

```

that this is a simple path. Second, since invoking FaPathD implies that t can be σ -reached within k hops during forward exploration, which in turn means that the initial σ predecessor of t can be reached within at most $k-1$ hops for the first time. Following this property, the entire reverse path can contain at most k vertices. Third, given $v_i = t$ and σ , based on FaPathD, we can obtain the carried sign of v_{i-1} (i.e., $\sigma^{v_{i-1}}$) is $\frac{\sigma}{\sigma_i}$. Then, we can obtain $\sigma^{v_{i-2}} = \frac{\sigma}{\sigma_i \cdot \sigma_{i-1}}$. Following this principle, we have $\sigma^{v_0} = \frac{\sigma}{\sigma_i \cdot \sigma_{i-1} \dots \sigma_1}$. Since $\sigma^{v_0} = \sigma^s = +$, we have $\sigma_i \cdot \sigma_{i-1} \dots \sigma_1 = \prod_{j=1}^i \sigma_j = \sigma$, where $\prod_{j=1}^i \sigma_j$ is the sign of the obtained path. The lemma holds. $\square$

PROOF OF THEOREM 4.6. Forward exploration. A vertex-sign pair (v, σ_v) is inserted into Q_1 only when $\text{reach}_v^{\sigma_v} = -1$ (Lines 10 and

16 of Algorithm 1), and $\text{reach}_v^{\sigma_v}$ is set to a non-negative value upon insertion. Hence, each of the at most $2|V|$ vertex-sign pairs is processed at most once. When (u, σ_u) is processed, all outgoing neighbors of u are scanned (Lines 7–16 of Algorithm 1). Since each vertex is processed at most twice (once per sign) and each outgoing edge of u is scanned once per processing of u , the total number of edge scans is bounded by $2|E|$. Together with the initialization of the reach records, forward exploration takes $O(|V| + |E|)$ time.

FaPathD. Each invocation visits at most k vertices, since it starts from t , which is σ -reached within k hops, and each step moves to the initial predecessor reached one hop earlier (Line 16 of Algorithm 1); moreover, only the vertices recorded in L are reset before returning (Algorithm 2). Hence, each invocation of FaPathD takes $O(k)$ time.

ExPathD-I. Constructing the candidate incoming neighbor sets $CN_{in}^{+/-}(\cdot)$ requires one scan of $N_{in}(v)$ for each vertex $v \in G$ under both signs, i.e., $O(|V| + |E|)$ time. Ordering the candidates by their hop counts takes $O(|V| + |E| + k)$ time with a counting sort, as the hop counts are integers in $[0, k]$ (our implementation uses a comparison-based sort, which adds only a logarithmic factor). The reverse search explores partial simple paths ending at t of length at most $k - 1$: the number of such partial paths is bounded by $\sum_{j=0}^{k-1} \delta_{in}^j = O(\delta_{in}^{k-1})$, and expanding one partial path scans at most δ_{in} candidate incoming neighbors, where $\delta_{in} = \max_{v \in V} |N_{in}(v)|$ is the largest in-degree in G . Hence, the reverse search takes $O(\delta_{in}^k)$ time, and ExPathD-I takes $O(|V| + |E| + \delta_{in}^k)$ time in total. Note that $\delta_{in} < 2|V|$, since each ordered vertex pair contributes at most one positive and one negative edge.

Overall bounds. Summing the three stages yields $O(|V| + |E| + \delta_{in}^k)$. For the space complexity, forward exploration maintains at most two reach records and two predecessor records per vertex ($O(|V|)$); the queues Q_0 and Q_1 hold at most $2|V|$ entries; FaPathD uses the visit array ($O(|V|)$) and the list L ($O(k)$); and ExPathD-I stores $CN_{in}^{+/-}(v)$ for each vertex v , whose total size is at most $2|E|$ since each incoming edge of v appears in $CN_{in}^+(v)$ and $CN_{in}^-(v)$ at most once each. Hence, the space complexity of LV0-I is bounded by $O(|V| + |E|)$. $\square$

PROOF OF LEMMA 4.8. Suppose $v \in SV_{d-1}^{\sigma_u}$. Based on the definition of signed shared vertices, v appears in all s - u paths with sign σ_u with length at most $d - 1$. Therefore, continuing to explore v along the current processed path will introduce a cycle. Besides, we note that *i*) during forward exploration, only the initial reach records for each vertex are computed and used in the subsequent valid path detection, and *ii*) if adding (v, σ_v) into Q_1 does not leads to update of $CSV^{\sigma_v}(v)$, the traversal induced by this path is equivalent to one already processed in a previous iteration for (v, σ_v) , and thus redundant. Therefore, the lemma holds. $\square$

PROOF OF LEMMA 4.9. Let d_0 be the depth at which (v, σ_v) is first σ_v -reached, via a pair (u, σ_u) . At that moment, $CSV^{\sigma_v}(v)$ is initialized as $SV_{d_0-1}^{\sigma_u}(u) \cup \{v\}$ (Line 18 of Algorithm 3). Since $SV_{d_0-1}^{\sigma_u}(u)$ is contained in the vertex set of a single s - u path of length at most $d_0 - 1$ and excludes s and t , it has at most $d_0 - 1$ elements, so $CSV^{\sigma_v}(v)$ has at most d_0 elements. By UR2, (v, σ_v) is inserted into Q_1 when it is first σ_v -reached and afterwards only when $CSV^{\sigma_v}(v)$ strictly shrinks (Lines 21–23). As $CSV^{\sigma_v}(v)$ is obtained by intersection, it

can only shrink, and it always contains v ; hence it shrinks at most $d_0 - 1$ times and (v, σ_v) is inserted into Q_1 at most d_0 times. Moreover, a pair inserted at depth d_0 is processed only in the iterations $d_0 + 1, \dots, k$, so it is processed at all only if $d_0 \leq k - 1$. Therefore, (v, σ_v) is processed at most $k - 1$ times during the forward exploration. $\square$

PROOF OF THEOREM 4.10. *Forward exploration.* By Lemma 4.9, each vertex-sign pair is processed at most $k - 1$ times. When (u, σ_u) is processed, all outgoing neighbors of u are scanned (Line 7 of Algorithm 3). Since each vertex is processed at most $2(k - 1)$ times, the total number of edge scans is bounded by $2(k - 1)|E|$. For each scanned neighbor, the check of UR1 (Line 9) is a binary search over $SV_{d-1}^{\sigma_u}(u)$ and takes $O(\log k)$ time; if not pruned, $CSV^{\sigma_v}(v)$ is either initialized by copying $SV_{d-1}^{\sigma_u}(u)$ (Line 18) or recomputed by a two-pointer merge (Line 21), both in $O(k)$ time since each set contains at most k elements. Hence, the edge scans take $O(k^2|E|)$ time in total. After each depth, $CSV^+(w)$ and $CSV^-(w)$ are copied to $SV_d^+(w)$ and $SV_d^-(w)$ for every vertex $w \in G$ (Lines 24–25), which takes $O(k|V|)$ time per depth and $O(k^2|V|)$ time over at most k depths. Together with the initialization of the reach records, forward exploration takes $O(k^2(|V| + |E|))$ time.

ExPathD-II. As in ExPathD-I, constructing and ordering the candidate incoming neighbor sets $CN_{in}^{+/-}(\cdot)$ takes $O(|V| + |E| + k)$ time. The reverse search explores partial simple paths ending at t of length at most $k - 1$, whose number is bounded by $O(\delta_{in}^{k-1})$. Expanding one partial path scans at most δ_{in} candidate incoming neighbors, and for each candidate, the SV -based pruning rule checks a set of at most k elements. Hence, the reverse search takes $O(k\delta_{in}^k)$ time, and ExPathD-II takes $O(|V| + |E| + k\delta_{in}^k)$ time in total.

Overall bounds. As in LV0-I, each invocation of FaPathD takes $O(k)$ time. Summing the three stages yields $O(k^2(|V| + |E|) + k\delta_{in}^k)$. For the space complexity, $SV_d^+(v)$ and $SV_d^-(v)$ for all $0 \leq d \leq k$ and $CSV^+(v)$ and $CSV^-(v)$ contain at most k elements each, contributing $O(k^2|V|)$ in total; the reach records and the visit array take $O(|V|)$; the queues Q_0 and Q_1 hold at most $2(k - 1)|V|$ entries by Lemma 4.9; the list L takes $O(k)$; and $CN_{in}^{+/-}(v)$ for all vertices v take at most $2|E|$ entries in total, as in ExPathD-I. Hence, the space complexity of LV0-II is bounded by $O(k^2|V| + |E|)$. $\square$

PROOF OF LEMMA 5.5. In the BFS-oriented traversal starting from vertex w during index construction, vertices are pushed into the queue Q in the order they are first discovered. Therefore, for vertex v with sign σ_v and its predecessor z with sign σ_z , (z, σ_z) is guaranteed to be added to Q before (v, σ_v) in Lines 1–14 of Algorithm 7. This means $\mathcal{L}_{in}^{\sigma_z}(z)$ is updated before $\mathcal{L}_{in}^{\sigma_v}(v)$ in Lines 15–18. Since only one (z, σ_z) is stored in Q , then only one entry will be added in $\mathcal{L}_{in}^{\sigma_z}(z)$, which contains z 's initial σ_z predecessor record. This entry must be appended to the end of $\mathcal{L}_{in}^{\sigma_z}(z)$. This also holds for all vertices with the corresponding carried sign in Q . Hence, in Lines 19–22, we can locate the entry that requires updating position information, and store the corresponding information for it. $\square$

THEOREM A.1. *The index-based query part (Lines 2–15 in HQA) takes $O(\eta k)$ time, where $\eta = |\mathcal{L}_{out}^+(s)| + |\mathcal{L}_{out}^-(s)| + |\mathcal{L}_{in}^+(t)| + |\mathcal{L}_{in}^-(t)|$. The overall time complexity of HQA is bounded by $O(\eta k + T_{LV0})$, where T_{LV0} denotes the time complexity of the invoked online*

method, i.e., $O(|V| + |E| + \delta_{in}^k)$ for LVO-I and $O(k^2(|V| + |E|) + k\delta_{in}^k)$ for LVO-II, with δ_{in} the largest in-degree in G .

PROOF OF THEOREM A.1. For each of the two sign combinations (σ_1, σ_2) with $\sigma_1 \cdot \sigma_2 = \sigma$, the merge join advances at least one of the two pointers at every vertex-wise comparison, so identifying all candidate entry pairs requires at most η pointer movements in total. Each candidate pair satisfying the hop constraint triggers at most one invocation of `IsSimplePath`. With the lazy-restoration implementation, the visit array is initialized only once before query processing, and each invocation marks and restores at most $k + 1$ traced vertices, i.e., $O(k)$ time per invocation rather than $O(|V|)$. Since the number of invocations is bounded by the number of matching entry pairs, which is at most η , the index-based query part takes $O(\eta k)$ time. When the index cannot resolve the query, the online method is invoked exactly once, so the overall time complexity of HQA is $O(\eta k + T_{LVO})$, where T_{LVO} is $O(|V| + |E| + \delta_{in}^k)$ for LVO-I and $O(k^2(|V| + |E|) + k\delta_{in}^k)$ for LVO-II. $\square$

THEOREM A.2. *Given a directed signed graph G and any query $Q(s, t, k, \sigma)$, HQA returns a correct result.*

PROOF OF THEOREM A.2. Soundness. If HQA returns true in Line 14 of Algorithm 4, then `IsSimplePath` has traced, by Lemma 5.5, the σ_1 path from s to v of length d and the σ_2 path from v to t of length d' corresponding to two matching entries with $\sigma_1 \cdot \sigma_2 = \sigma$ (Line 3) and $d + d' \leq k$ (Line 11), and has found no repeated vertex on their concatenation; the concatenation is thus a valid s - t path. If HQA returns the result of LVO-based methods in Line 18, the answer is correct since LVO-based methods are exact.

Completeness. We show that if a valid path exists, then $F_{touch} = \text{true}$, so that HQA does not return false in Line 19. It suffices to prove the following claim for any τ path P from x to y (not necessarily simple) on which the highest-ranked vertex occurs only once, which is the case for every simple path: the SRL contains entries $(v, a, \cdot, \cdot) \in \mathcal{L}_{out}^\alpha(x)$ and $(v, b, \cdot, \cdot) \in \mathcal{L}_{in}^\beta(y)$ for some vertex v with $\alpha \cdot \beta = \tau$ and $a + b \leq |P|$. Indeed, applying the claim to P with $x = s$, $y = t$ and $\tau = \sigma$, the merge join-based scanning in Lines 5-10 reaches the common vertex v for $\sigma_1 = \alpha$ unless HQA has already returned true, since each label set holds at most one entry per vertex and the entries are sorted by rank, and $a + b \leq |P| \leq k$ sets F_{touch} to true in Line 12. We prove the claim by induction on the rank of the highest-ranked vertex v on P , from the highest rank downwards. Since v occurs only once on P , P splits into the subpaths P_1 from x to v and P_2 from v to y ; let α and β be their signs, so that $\alpha \cdot \beta = \tau$. Consider the backward traversal rooted at v (Algorithm 7 with N_{in}). Since every vertex on P_1 other than v is ranked lower than v , none of them is skipped in Line 7, and the vertex-sign pairs along P_1 are enqueued at depth no larger than their distance to v along P_1 (Lines 9-12), unless one of them is pruned by `IsDerivable` in Line 10. Suppose first that a pair is pruned, and let (u, γ) be the pruned pair closest to v along P_1 , reached at depth no larger than the length of the subpath of P_1 from u to v . By `IsDerivable`, the existing labels represent a route from u to v through a hub h with sign γ and length no larger than this depth. Here $h \neq v$, since the label sets enumerated by `IsDerivable` are those on the side written by the current traversal, and the entries with hub v on this side are recorded only in its labeling phase; thus, h is ranked higher than

v by Lemma 5.2. Replacing the subpath of P_1 from u to v with this route produces a τ path P' from x to y with $|P'| \leq |P|$, on which h is the highest-ranked vertex and occurs only once: every vertex of P is ranked lower than h , and the two label-recorded paths forming the route contain, apart from their endpoint h , only vertices ranked lower than h by Line 7. The claim for P' holds by the induction hypothesis and implies the claim for P , as $a + b \leq |P'| \leq |P|$. The base case, where v is the highest-ranked vertex of G , never falls into this case, since the label sets of v contain no other vertex by Lemma 5.2 and `IsDerivable` never prunes. Suppose now that no pair along P_1 is pruned. Then x receives the entry $(v, a, \cdot, \cdot) \in \mathcal{L}_{out}^\alpha(x)$ with $a \leq |P_1|$ in the labeling phase (Lines 15-17); if P_1 is empty, i.e., $v = x$, this is the entry recorded for the root itself with $a = 0$, since the root is enqueued in Line 1 and labeled in Lines 15-17 like any other pair. The forward traversal rooted at v along P_2 is symmetric: if a pair along P_2 is pruned, the same rerouting argument proves the claim; otherwise, y receives the entry $(v, b, \cdot, \cdot) \in \mathcal{L}_{in}^\beta(y)$ with $b \leq |P_2|$, which together with the entry of x gives $\alpha \cdot \beta = \tau$ and $a + b \leq |P_1| + |P_2| = |P|$. Hence, the claim holds. $\square$

THEOREM A.3. *Given a directed signed graph $G = (V, E)$, ICA takes $O(|V|(|V| + \lambda|E|))$ time to construct the SRL, where λ is the maximum number of entries in any signed in-label or out-label set within the SRL.*

PROOF OF THEOREM A.3. Consider one invocation of the procedure `BuildSignLabel` rooted at a vertex w . Each vertex-sign pair is enqueued at most once (Lines 9-12 of Algorithm 7), and processing an enqueued pair (u, σ_u) scans the neighbors of u along the traversal direction once. Since each vertex is enqueued at most twice (once per sign), the total number of edge scans is bounded by $2|E|$. A scan that reaches a pair (v, σ_v) with $\text{reach}_v^{\sigma_v} = -1$ invokes `IsDerivable` (Line 10). Note that a pair found derivable keeps $\text{reach}_v^{\sigma_v} = -1$ and is checked again when it is scanned later; hence, the number of invocations of `IsDerivable` is bounded by the number of edge scans, i.e., $2|E|$. In `IsDerivable` (Algorithm 8), the entries of $\mathcal{L}_{in}^+(v)$ and $\mathcal{L}_{in}^-(v)$ are enumerated, and for each entry the existence of a matching entry in $\mathcal{L}_{out}^{\sigma_v}(w)$ is tested (Line 5). In our implementation, $\mathcal{L}_{out}^+(w)$ and $\mathcal{L}_{out}^-(w)$ are loaded into an array indexed by vertex before the traversal starts, which takes $O(\lambda)$ time, so that each test in Line 5 takes $O(1)$ time and one invocation of `IsDerivable` takes $O(\lambda)$ time. The labeling phase (Lines 15-22) appends at most $2|V|$ entries and takes $O(|V|)$ time. Hence, one invocation of `BuildSignLabel` takes $O(|V| + \lambda|E|)$ time. Since `BuildSignLabel` is invoked twice for each vertex $w \in V$ (once along each direction), ICA takes $O(|V|(|V| + \lambda|E|))$ time. $\square$

THEOREM A.4. *Given a directed signed graph G and the SRL-index of G , the total label size is $\sum_{v \in G} (|\mathcal{L}_{in}^+(v)| + |\mathcal{L}_{in}^-(v)| + |\mathcal{L}_{out}^+(v)| + |\mathcal{L}_{out}^-(v)|)$. The space complexity is bounded by $O(|V|^2)$.*

PROOF OF THEOREM A.4. For each vertex, it will store at most $|V| - 1$ vertices and at most four different label sets for each vertex. Therefore, the space complexity is bounded by $O(|V|^2)$. $\square$

LEMMA A.5. *Let $P = \langle s = w_0, w_1, \dots, w_l = v \rangle$ be a simple σ_v path with $l \leq k$. For every $j < l$, the expansion from (w_j, σ_{w_j}) to w_{j+1} is not rejected by UR1 in Line 9 of Algorithm 3. Consequently, $SV_1^{\sigma_v}(v)$ is contained in the vertex set of P .*

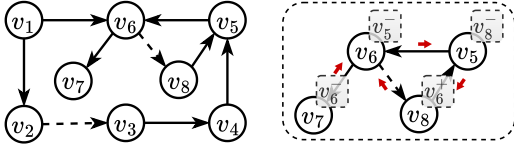


Figure 8: Another directed signed graph (solid lines denote the positive edges and dotted line denote the negative edges) FaPathD cannot find a valid path under query $Q(v_1, v_7, 6, -)$

PROOF. We prove by induction on j that $SV_j^{\sigma w_j}(w_j) \subseteq V_j := \{w_1, \dots, w_j\} \setminus \{t\}$. For $j = 1$, UR1 checks $w_1 \in SV_0^+(s) = \emptyset$ and never prunes, and $CSV^{\sigma w_1}(w_1) \subseteq \{w_1\}$ by Lines 18 and 21.

Assume the claim for j . Let $j' \leq j$ be the last iteration at which $CSV^{\sigma w_j}(w_j)$ changed; by UR2, (w_j, σ_{w_j}) is inserted into Q_1 at iteration j' and is therefore processed at iteration $j' + 1 \leq j + 1$, at which UR1 tests $w_{j+1} \in SV_{j'}^{\sigma w_j}(w_j) = SV_j^{\sigma w_j}(w_j) \subseteq V_j$. Since P is simple, $w_{j+1} \notin V_j$, so the traversal is not pruned. Note that other traversals reaching (w_j, σ_{w_j}) only shrink $SV_j^{\sigma w_j}(w_j)$ further, so the inclusion $SV_j^{\sigma w_j}(w_j) \subseteq V_j$ is preserved.

Lines 18 and 21 then give $CSV^{\sigma w_{j+1}}(w_{j+1}) \subseteq SV_j^{\sigma w_j}(w_j) \cup \{w_{j+1}\} \subseteq V_{j+1}$. As CSV only shrinks, $SV_{j+1}^{\sigma w_{j+1}}(w_{j+1}) \subseteq V_{j+1}$, which completes the induction. $\square$

LEMMA A.6. *Given a directed signed graph G , a query $Q(s, t, k, \sigma)$ and a backtracking instance (s, w, σ_w, h, k) in ExPathD-II. Suppose the current processed incoming neighbor is $(d_v, v, \sigma_{vw}) \in CN_{in}^{\sigma w}(w)$, we present two pruning rules based on the hop constraint (PR1-PR2) and two pruning rules based on the simple path constraint (PR3-PR4).*

PROOF OF LEMMA A.6. Let $d' = k - h - 1$ denote the number of hops still available from s to v : after the search moves from w to v the depth becomes $h + 1$, so every s - v path that completes the current partial path has length at most d' . For PR1, by the hop constraint, the resulting s - t path has length at most k . When $h = k - 1$, only one edge remains, so the candidate v must be s itself and every other candidate can be rejected. For PR2, by Lemma A.5, no traversal along a simple path is pruned by UR1, so (v, σ_v) is σ_v -reached no later than the length of the shortest simple σ_v path from s to v . Hence d_v is a lower bound on the length of every simple σ_v path from s to v , and the resulting s - t path has length at least $h + 1 + d_v$. Therefore, if $h + 1 + d_v > k$, there is no valid path. For PR3, if v has already been visited in the current backtracking path from t to w , extending this path to v would visit v twice, violating the simple path constraint. For PR4, suppose that there exists a vertex $u \in SV_{d'}^{\sigma v}(v)$ that has already been visited in the current backtracking path from t to w , then there exists a partial path from t to u already established in the current search context. By Lemma A.5, $SV_{d'}^{\sigma v}(v)$ is contained in the vertex set of every simple σ_v path from s to v of length at most d' , so u must also appear in any valid path from s to v . Concatenating these two paths implies that u would be visited twice in a single s - t path, forming a cycle. This contradicts the simple path constraint. Hence, the traversal through v cannot yield a valid solution and can be safely pruned. $\square$

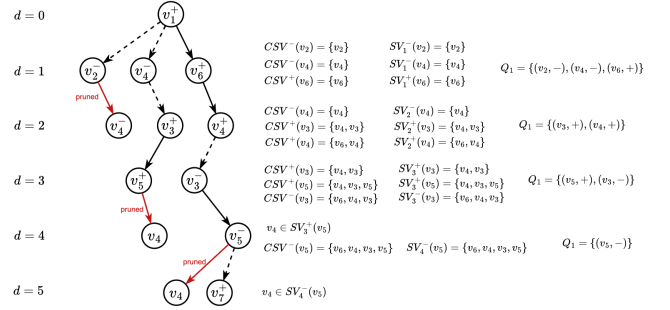


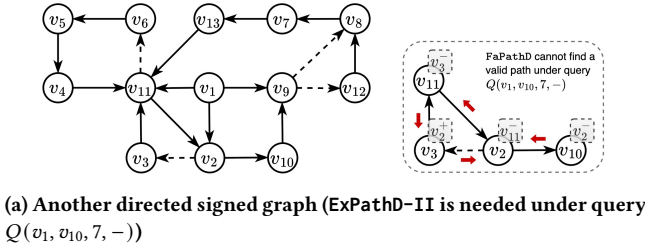
Figure 9: Example of forward exploration under query $(v_1, v_7, 5, +)$ in directed signed graph of Figure 1

A.3 Examples

Example A.7 (Example of false negative issue of FaPathD). Consider another directed signed graph in Figure 8 and a query $Q(v_1, v_7, 6, -)$. When the target vertex v_7 is negative-reached for the first time, we first invoke FaPathD to detect the valid path. We illustrate this scenario on the right of Figure 8. During the reverse traversal, we sequentially visit v_7^- , v_6^- , v_5^- , v_8^- and v_6^+ . Since vertex v_6 is revisited in this exploration, i.e., a ‘cycle’ is detected, FaPathD returns false. However, a valid simple path from v_1 to v_7 does exist, i.e., $v_1^+ \xrightarrow{+} v_2^+ \xrightarrow{-} v_3^- \xrightarrow{+} v_4^- \xrightarrow{+} v_5^- \xrightarrow{+} v_6^- \xrightarrow{+} v_7^-$.

Example A.8 (Example of forward exploration in LVO-II). Consider the DSG in Figure 1 and a query $Q(v_1, v_7, 5, +)$. We illustrate the computed SV^+ / SV^- for some vertices at different depths in Figure 9. The traversals pruned by Lemma 4.8 are highlighted in red in Figure 9. For example, suppose the currently explored vertex is v_5 , explored via the traversal $v_1^+ \xrightarrow{-} v_4^- \xrightarrow{-} v_3^+ \xrightarrow{+} v_5^+$. When examining v_5 ’s outgoing neighbor $(v_4, +)$ at depth $d = 4$, this exploration can be safely pruned due to UR1, since $v_4 \in SV_3^+(v_5) = \{v_4, v_3, v_5\}$. We also illustrate the computed Q_1 at each search depth in this figure. For example, consider the case where $Q_0 = \{(v_2, -), (v_4, -), (v_6, +)\}$ at depth $d = 2$. Suppose we are currently processing the vertex-sign pair $(v_2, -)$, and we explore v_2 ’s outgoing neighbor. Since $(v_4, +)$ is the only outgoing neighbor of v_2 , the traversal implies that v_4 is negative-reached via this path. However, we observe that this is not the first time v_4 is negative-reached. In particular, we have $CSV^-(v_4) = \{v_4\}$ and the newly traversed path information is $SV_1^-(v_2) \cup \{v_4\} = \{v_2, v_4\}$. Their intersection is $CSV^-(v_4) \cap (SV_1^-(v_2) \cup \{v_4\}) = CSV^-(v_4)$, which indicates that the newly discovered traversal does not change $CSV^-(v_4)$, and thus does not provide additional information. Hence, $(v_4, -)$ will not be added into Q_1 due to UR2.

Example A.9. Given a directed signed graph in Figure 10(a) and the query $Q(v_1, v_{10}, 7, -)$, FaPathD cannot identify a valid path, as illustrated on the right side of Figure 10(a). Consequently, ExPathD-II needs to be invoked to perform the valid path detection. Figure 10(b)-(c) illustrate the initial reach records and the candidate incoming neighbor sets for each vertex. Note that PR1-PR3 are applied by ExPathD-I and ExPathD-II alike, whereas PR4 is available only to ExPathD-II, which maintains the signed shared vertices during forward exploration.



	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}	v_{11}	v_{12}	v_{13}
pre^+	v_1^+	v_1^+	v_2^-	v_5^+	v_6^+	v_{11}^-			v_1^+	v_1^+			
$reach^+$	0	1	7	6	5	4	-1	-1	1	-1	1	-1	-1

	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}	v_{11}	v_{12}	v_{13}
pre^-		v_{11}^-	v_2^+	v_5^-	v_6^-	v_{11}^+	v_8^-	v_9^+		v_2^-	v_3^-	v_9^+	v_7^-
$reach^-$		6	2	4	3	2	3	2	-1	7	3	2	4

(b) Initial reach records

	v_1	v_2	v_3	v_4	v_5	v_6	v_7
$CN_{in}^+(*)$	$\{(0, v_1, +)\}$	$\{(6, v_2, -)\}$	$\{(5, v_5, +)\}$	$\{(4, v_6, +)\}$	$\{(3, v_{11}, -)\}$		
$CN_{in}^-(*)$	$\{(3, v_{11}, +)\}$	$\{(1, v_2, -)\}$	$\{(3, v_5, +)\}$	$\{(2, v_6, +)\}$	$\{(1, v_{11}, -)\}$	$\{(2, v_8, +)\}$	

	v_8	v_9	v_{10}	v_{11}	v_{12}	v_{13}
$CN_{in}^+(*)$	$\{(0, v_1, +)\}$	$\{(1, v_2, +)\}$	$\{(0, v_1, +)\}$	$\{(6, v_4, +)\}$	$\{(7, v_3, +)\}$	
$CN_{in}^-(*)$	$\{(1, v_9, -)\}$	$\{(7, v_{10}, +)\}$	$\{(6, v_2, +)\}$	$\{(2, v_3, +)\}$	$\{(4, v_4, +)\}$	$\{(1, v_9, -)\}$

(c) Candidate incoming neighbor sets

Figure 10: An example of ExPathD-II

In Figure 11, we illustrate an example where PR2 can be applied. Suppose the current reverse traversal visits vertices $v_{10}, v_2, v_{11}, v_{13}, v_7, v_8$. At this point, the backtracking instance is $(v_1, v_8, -, h = 5, k = 7)$. We examine the candidate incoming neighbors $CN_{in}^-(v_8) = \{(1, v_9, -), (2, v_{12}, +)\}$. The candidate incoming neighbors are examined in ascending order of their hop counts, so v_9 is considered first; if the search were to proceed through v_{12} , we observe that $reach_{v_{12}}^- = 2$, thus $h + reach_{v_{12}}^- + 1 = 8 > k$, violating the hop constraint. Therefore, the current traversal through v_{12} is pruned due to PR2.

We then proceed to explore v_9 , whose backtracking instance is $(v_1, v_9, +, h = 6, k = 7)$ and whose candidate incoming neighbor set is $CN_{in}^+(v_9) = \{(0, v_1, +)\}$. This is the only state with $h = k - 1$ in this instance, and PR1 accepts v_1 since $v_1 = s$. In general, PR1 is the special case of PR2 at $h = k - 1$: every candidate other than s has a hop count of at least one and is therefore rejected, and PR1 detects this without computing the sum.

Example A.10. Following Example A.9 with query $Q(v_1, v_{10}, 7, -)$, we illustrate two examples where PR3 and PR4 can be applied during traversal. Suppose the current reverse traversal reaches vertices v_{10}, v_2, v_{11}, v_3 , and the current backtracking instance is $(v_1, v_3, -, h = 3, k = 7)$. Next we proceed to explore v_3 's candidate incoming neighbor, i.e., $CN_{in}^-(v_3) = \{(1, v_2, -)\}$. We find that v_2 has already appeared in the current explored path. Therefore, the current traversal is pruned due to PR3. Note that this is the behaviour of ExPathD-I; in ExPathD-II, the candidate v_3 is already

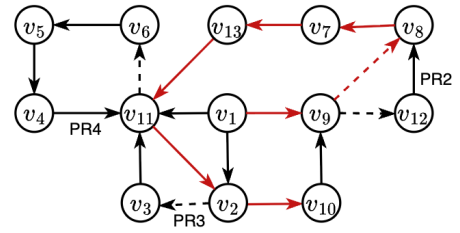


Figure 11: An example of PR2-PR4 used in ExPathD-II under query $Q(v_1, v_{10}, 7, -)$

rejected at v_{11} by PR4, since $SV_4^-(v_3) = \{v_2, v_3\}$ contains the visited vertex v_2 .

Suppose the current reverse traversal reaches vertices v_{10}, v_2, v_{11} , and the current backtracking instance is $(v_1, v_{11}, -, h = 2, k = 7)$. We now explore the candidate $(4, v_4, +) \in CN_{in}^-(v_{11})$. When processing v_4 , given $h = 2$, the remaining allowable search depth from v_4 to v_1 is $k - h - 1 = 4$. Based on the definition of signed shared vertices, we have $SV_4^-(v_4) = \{v_{11}, v_6, v_5, v_4\}$. Since $v_{11} \in SV_4^-(v_4)$ is already marked as visited in the current explored path, the current traversal is safely pruned due to PR4. This branch also shows the benefit of PR4 over the pruning rules available to ExPathD-I. Without PR4, the traversal would proceed from v_4 to v_5 and then to v_6 , and only at v_6 would the candidate v_{11} be rejected by PR3, after three additional levels of search. PR4 rejects v_4 at once, since $SV_4^-(v_4)$ already reveals that every negative path from v_1 to v_4 of length at most 4 passes through the visited vertex v_{11} .

A.4 Omitted experiments

Query selection of Exp-1. The query set for each dataset is generated based on the structural characteristics of real-world patterns observed in large-scale logs (e.g., [8]). Specifically, (1) source vertices sampled from the subset of vertices $V_{20\%}$, which is the top 20% of all vertices ranked by total degree (the sum of in-degree and out-degree). Each vertex v is selected with a probability $p(v)$ proportional to its total degree, i.e., $p(v) = \frac{|N_{in}(v)| + |N_{out}(v)|}{\sum_{u \in V_{20\%}} (|N_{in}(u)| + |N_{out}(u)|)}$. (2) For target vertices, we employ a hybrid strategy to reflect diverse query intents found in real-world scenarios: i) 35% short-range targets, where each target vertex t is selected via a random walk of 2 hops from the source vertex. ii) 35% target vertices are sampled from the top 1% of vertices ranked by in-degree. iii) 15% target vertices are sampled from the top 20% high total degree vertices. and iv) 15% target vertices are sampled uniformly at random from all vertices V . (3) We also ensure a balanced number of positive and negative query constraints. These settings allow our workload to capture the star-shaped source topology, short-range navigational locality, and skewed structural convergence characteristic of real-world query patterns.

Exp-9: Total query time with varying negative edge ratio. We also evaluate the performance of the proposed algorithms by varying the negative edge ratio on D4 and D8. For D4, a real-world directed signed graph with about 15% negative edges, we generate six variants by randomly flipping negative edges to positive to reach a ratio of 10%, and positive edges to negative to reach ratios from 20% to 60%. To further assess performance on larger scales, we generate six variants for D8 by marking a uniformly

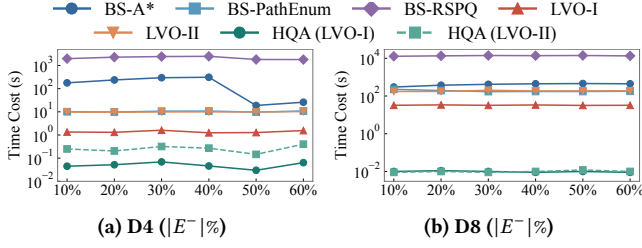


Figure 12: Total query time vs. negative edge ratios.

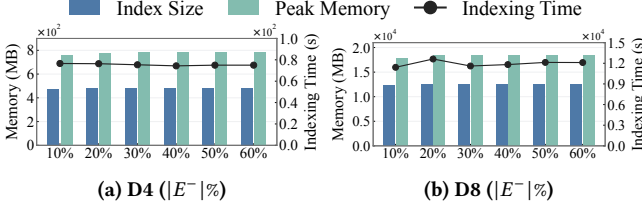


Figure 13: Index construction vs. negative edge ratio.

random 10% to 60% of the edges as negative. Figure 12 reports the total query time under the workload of Exp-1 at $k = 6$; BS-BiBFS, BS-Hansen and BS-BMAS are omitted since they do not complete on these datasets. As observed, the query time of LVO-I and LVO-II is almost unaffected by the negative edge ratio, varying by at most a factor of 1.16 across the six variants except for LVO-I on D4, where the factor is 1.3. HQA answers the 1000 queries within 0.41 seconds on D4 and 0.02 seconds on D8 at every ratio. By contrast, BS-A* fluctuates by more than an order of magnitude on D4, from 18.8 to 310 seconds, and BS-RSPQ remains the slowest at every ratio.

Exp-10: Total query time by varying degree. In this experiment, we evaluate the sensitivity of all algorithms to the degrees of the query source and target vertices. We rank all vertices on D8 by the product of their in-degree and out-degree, sorted in descending order, and partition them into five equal-sized groups. For each group, we generate a query set containing 1000 queries, where both the source vertex s and target vertex t are randomly sampled from the vertices within that specific group. We also ensure an approximately equal number of positive and negative query constraints. Figure 14(a) reports the total response time of all algorithms for each query set. The x-axis labels De1 to De5 correspond to the vertex groups in descending order, ranging from the top 20% (De1) to the bottom 20% (De5). Note that, other parameters are kept at their default values (e.g., $k = 6$). As observed, LVO-based algorithms consistently outperform baselines in most settings. In the fifth group (De5), BS-A* and BS-PathEnum outperform the LVO-based algorithms. This is because the majority of queries in De5 return false, and both baselines reject such queries with a backward search from t : BS-A* prunes them by its distance lower bound, and BS-PathEnum obtains an almost empty active vertex set. In contrast, the LVO-based algorithms complete the forward exploration from s before concluding false, and LVO-II additionally maintains its shared-vertex sets throughout. Furthermore, HQA consistently achieves the best performance for all tested degree groups, maintaining a speedup of several orders of magnitude.

Exp-11: Total query time by varying distance. We also conduct experiments on D8 by varying distance l between the query

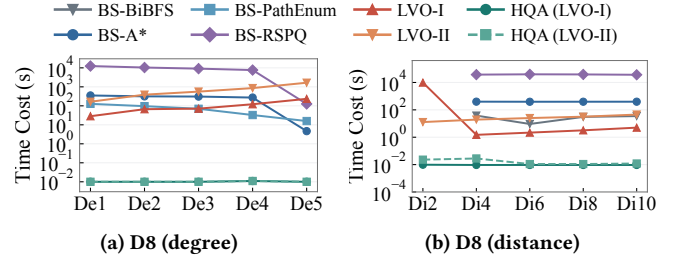


Figure 14: Total query time vs. query vertex degree/distance.

source and target vertices. For each $l \in \{2, 4, 6, 8, 10\}$, we generate a group of 1000 queries (labeled Di2–Di10) with balanced positive and negative sign constraints. The hop constraint is fixed at $k = 12$ to provide sufficient search depth. Figure 14(b) reports the total response time of each algorithm across these groups. BS-PathEnum fails to finish within the time limit on all five groups. LVO-I significantly outperforms BS-A* by up to two orders of magnitude. For instance, at $l = 4$, BS-A* requires 396 seconds to return the result, whereas LVO-I completes the same task in only 1.5 seconds. Additionally, while BS-BiBFS even suffers from memory exhaustion on small graphs for certain queries that involve dense local structures, it exhibits stable performance in this experiment. This is mainly because query pairs are sampled randomly, and a longer path distance x does not necessarily correlate with large vertex degrees. Consequently, the search frontiers remain manageable within the memory limits. Notably, on Di2, LVO-II is the only online method that completes within a reasonable time (13 seconds); BS-BiBFS runs out of memory, all other baselines exceed the 12-hour limit, and even LVO-I takes 2.83 hours. This demonstrates the stability of LVO-II. Not surprisingly, HQA consistently maintains the lowest query cost across all groups, demonstrating its robust efficiency regardless of the distance between query vertices.

Exp-12: Indexing cost vs. negative edge ratios. We also evaluate the index construction cost by varying the negative edge ratio on D4 and D8, using the variants of Exp-9. Figure 13 reports the index size, peak memory, and construction time across these variants. As observed, all three metrics are largely insensitive to the negative edge ratio: the index size and the peak memory vary by less than 4% on both datasets, and the construction time by less than 11%, demonstrating the robustness of the index construction algorithm to the sign distribution.

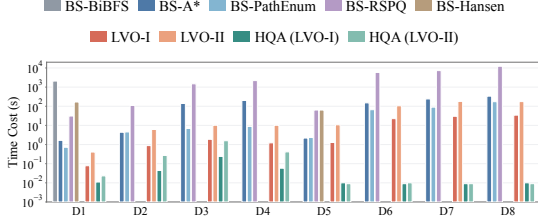


Figure 15: Total query time over all datasets with 100% negative sign constraints (Exp-1).

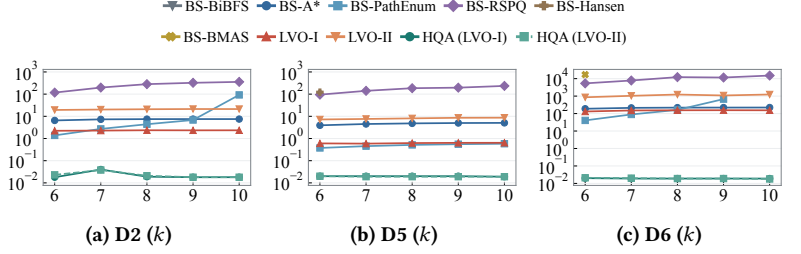


Figure 16: Total query time for 2000 random mix-sign queries with varying k on D2, D5 and D6 (Exp-3).

Table 5: Exp-5: frequency and cost of exact detection on the workload of Exp-1 (1000 queries per dataset).

Alg.	LVO-I					LVO-II					HQA (LVO-II)							
k	$\frac{C1}{C1+C2}$	C2	C4	Total (s)	$\frac{ExpPathD-I}{Total}$	$\frac{C1}{C1+C2}$	C2	C4	Total (s)	$\frac{ExpPathD-II}{Total}$	#ISP	#LVO-II	$\frac{C1}{C1+C2}$	C2	C4	Total (s)	$\frac{LVO-II}{Total}$	$\frac{ExpPathD-II}{Total}$
D2																		
6	99.55%	4	0	0.744	14.49%	99.89%	1	0	5.52	0.64%	885	2	100%	0	0	0.122	91.69%	0%
7	99.55%	4	0	0.757	14.30%	99.89%	1	0	5.59	0.63%	887	2	100%	0	0	0.121	91.90%	0%
8	99.55%	4	0	0.783	14.09%	99.89%	1	0	5.61	0.63%	887	2	100%	0	0	0.130	92.44%	0%
9	99.55%	4	0	0.769	14.18%	99.89%	1	0	6.29	0.56%	887	2	100%	0	0	0.141	93.03%	0%
10	99.55%	4	0	0.813	13.77%	99.89%	1	0	6.78	0.56%	887	2	100%	0	0	0.148	93.36%	0%
D3																		
6	99.21%	7	0	1.66	51.03%	99.89%	1	0	7.83	0.62%	884	15	100%	0	0	0.986	98.95%	0%
7	99.21%	7	0	61.4	98.64%	99.89%	1	0	7.81	0.64%	884	15	100%	0	0	0.953	98.90%	0%
8	99.21%	7	0	4753	99.98%	99.89%	1	0	7.76	0.64%	884	15	100%	0	0	0.969	98.92%	0%
9	INF	INF	INF	INF	INF	99.89%	1	0	7.88	0.64%	884	15	100%	0	0	0.974	98.93%	0%
10	INF	INF	INF	INF	INF	99.89%	1	0	7.98	0.63%	884	15	100%	0	0	0.988	98.94%	0%
D4																		
6	99.67%	3	0	1.26	11.70%	99.89%	1	0	9.56	0.62%	915	5	100%	0	0	0.319	96.86%	0%
7	99.67%	3	0	1.15	11.54%	99.89%	1	0	9.56	0.61%	915	5	100%	0	0	0.300	96.57%	0%
8	99.67%	3	0	1.15	11.53%	99.89%	1	0	9.97	0.58%	915	6	100%	0	0	0.400	97.48%	0%
9	99.67%	3	0	1.15	11.70%	99.89%	1	0	9.99	0.56%	915	6	100%	0	0	0.396	97.45%	0%
10	99.67%	3	0	1.14	11.52%	99.89%	1	0	9.26	0.53%	915	6	100%	0	0	0.406	97.51%	0%
D5																		
6	100%	0	0	1.16	0%	100%	0	0	11.4	0%	615	0	100%	0	0	0.0100	0%	0%
7	100%	0	0	1.42	0%	100%	0	0	11.7	0%	621	0	100%	0	0	0.0095	0%	0%
8	100%	0	0	1.38	0%	100%	0	0	12.3	0%	626	0	100%	0	0	0.0094	0%	0%
9	100%	0	0	1.27	0%	100%	0	0	14.4	0%	627	0	100%	0	0	0.0094	0%	0%
10	100%	0	0	1.27	0%	100%	0	0	13.1	0%	628	0	100%	0	0	0.0095	0%	0%

Table 6: Exp-5: frequency and cost of exact detection on the uniform workload (10,000 queries per dataset).

Alg.	LVO-I					LVO-II					HQA (LVO-II)								
k	$\frac{C1}{C1+C2}$	C2	C4	Total (s)	$\frac{ExpPathD-I}{Total}$	$\frac{C1}{C1+C2}$	C2	C4	Total (s)	$\frac{ExpPathD-II}{Total}$	#ISP	#LVO-II	$\frac{C1}{C1+C2}$	C2	C4	Total (s)	$\frac{LVO-II}{Total}$	$\frac{ExpPathD-II}{Total}$	
D2																			
6	99.88%	4	0	11.0	0.93%	99.97%	1	0	97.8	0.04%	3471	0	100%	0	0	0.0967	0%	0%	
7	99.89%	4	0	11.9	0.85%	99.97%	1	0	109	0.03%	3694	0	100%	0	0	0.0940	0%	0%	
8	99.89%	4	0	12.0	0.85%	99.97%	1	0	113	0.03%	3746	0	100%	0	0	0.0938	0%	0%	
9	99.89%	4	0	12.5	0.93%	99.97%	1	0	113	0.03%	3753	0	100%	0	0	0.0932	0%	0%	
10	99.89%	4	0	12.3	0.90%	99.97%	1	0	115	0.03%	3758	0	100%	0	0	0.0930	0%	0%	
D3																			
6	99.81%	4	0	7.77	10.70%	99.95%	1	0	65.8	0.08%	2073	0	100%	0	0	0.0930	0%	0%	
7	99.81%	4	0	71.4	90.49%	99.95%	1	0	66.2	0.08%	2082	0	100%	0	0	0.0924	0%	0%	
8	99.81%	4	0	5851	99.88%	99.95%	1	0	66.9	0.09%	2084	0	100%	0	0	0.0924	0%	0%	
9	INF	INF	INF	INF	INF	99.95%	1	0	66.4	0.09%	2084	0	100%	0	0	0.0926	0%	0%	
10	INF	INF	INF	INF	INF	99.95%	1	0	66.9	0.09%	2084	0	100%	0	0	0.0926	0%	0%	
D4																			
6	100%	0	0	29.2	0%	100%	0	0	270	0%	3223	0	100%	0	0	0.0950	0%	0%	
7	100%	0	0	32.3	0%	100%	0	0	298	0%	3335	0	100%	0	0	0.0935	0%	0%	
8	100%	0	0	34.4	0%	100%	0	0	314	0%	3365	0	100%	0	0	0.0948	0%	0%	
9	100%	0	0	30.7	0%	100%	0	0	318	0%	3373	0	100%	0	0	0.0946	0%	0%	
10	100%	0	0	31.4	0%	100%	0	0	327	0%	3374	0	100%	0	0	0.0939	0%	0%	
D5																			
6	100%	0	0	3.26	0%	100%	0	0	39.8	0%	690	0	100%	0	0	0.0989	0%	0%	
7	100%	0	0	3.92	0%	100%	0	0	42.9	0%	733	0	100%	0	0	0.0984	0%	0%	
8	100%	0	0	3.88	0%	100%	0	0	45.3	0%	754	0	100%	0	0	0.0977	0%	0%	
9	100%	0	0	3.93	0%	100%	0	0	48.3	0%	759	0	100%	0	0	0.0991	0%	0%	
10	100%	0	0	3.90	0%	100%	0	0	49.4	0%	763	0	100%	0	0	0.0978	0%	0%	